

MVC (Model View Control)

What is Spring MVC ?

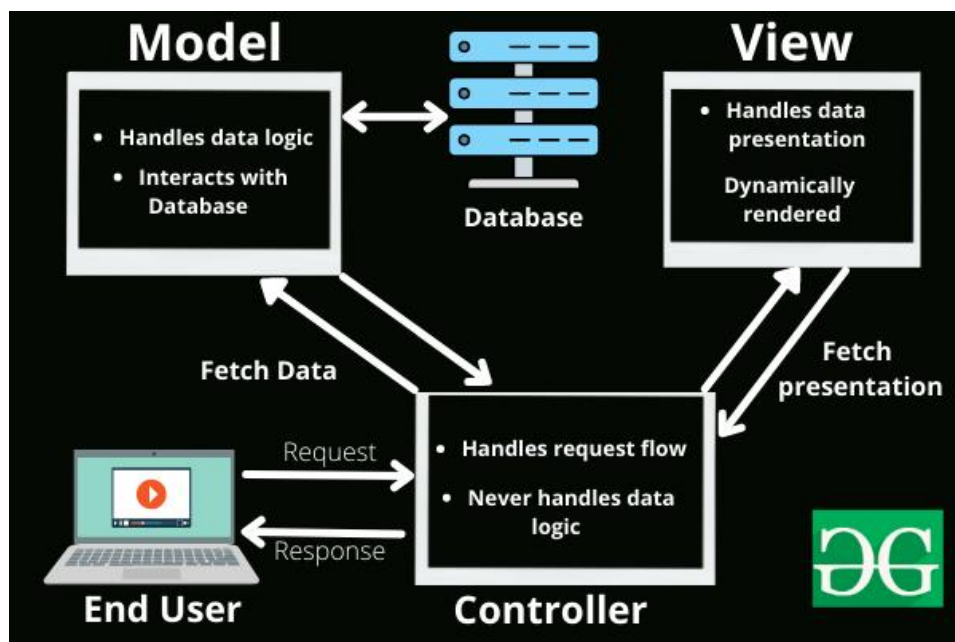
- Spring MVC uses the Model-View-Controller pattern to structure web applications.
- It has a central DispatcherServlet, which receives all incoming HTTP requests.
- DispatcherServlet routes each request to the correct controller method.
- The @Controller annotation marks a class as a Spring MVC controller.

Features of MVC :-

1. Separates business, UI, and input logic.
2. Gives full control over HTML and URLs.
3. Provides clean and searchable URL mapping.
4. Supports Test-Driven Development (TDD).

Components of MVC :-

The MVC framework includes the following 3 components:



- 1. Model:** The Model encapsulates the application data.
 - Manages all data-related logic.
 - Performs CRUD operations and applies business rules.
 - Talks to the database and returns data to the Controller.
 - **Example:** Stores and retrieves book details like title, price, and stock.
- 2. View:** The View renders the model data and generates HTML output that the client's browser can interpret.
 - Handles all UI logic of the application.

- Displays data provided by the Controller.
- Updates the user interface when data changes.
- **Example:** Shows book lists, book details, and search input fields.

3. Controller: The Controller processes the user requests and passes them to the view for rendering.

- Acts as a link between the View and Model.
- Doesn't handle data logic; instructs the Model what to do.
- Processes business logic and user requests.
- Updates the Model and selects the View for output.
- **Example:** Handles book search, add-to-cart, or checkout actions.

What is Dispatcher Servlet ?

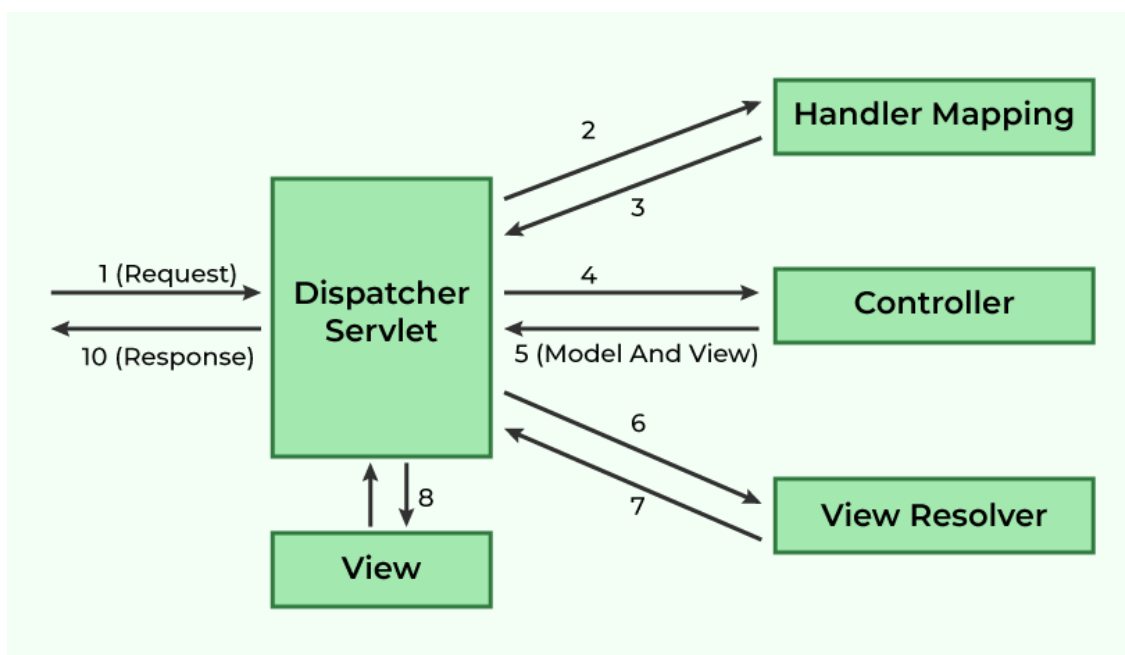
The DispatcherServlet is the central component of Spring MVC:

- Acts as the Front Controller for all HTTP requests.
- Determines the appropriate controller to handle a request.
- Dispatches the request to the selected controller.
- Resolves the view using the View Resolver and renders the response.

Steps to Setting Up DispatcherServlet :-

1. Add the Spring MVC dependency in your **pom.xml**.
2. Create the **web.xml** file in WEB-INF to configure the DispatcherServlet.
3. Define the DispatcherServlet inside web.xml with a servlet name (e.g., "spring").
4. Map the servlet to a URL pattern like / or *.do.
5. Create the Spring configuration file (e.g., spring-servlet.xml).
6. Enable component scanning to detect controllers.
7. Configure view resolver to locate JSP files.
8. Restart the server — DispatcherServlet will now handle all incoming requests.

Spring MVC Flow Diagram :-



- DispatcherServlet intercepts incoming requests.
- Retrieves the handler mapping and forwards the request to the controller.

- The controller processes the request and returns a ModelAndView object.
- DispatcherServlet uses the view resolver to render the appropriate view.

Implementation of Spring MVC Application :-

i.e. addition of two numbers.

Create a Maven-based Spring MVC project. Ensure the required folder structure and server configuration are in place.

Step 1: Add dependencies in pom.xml

Include the following dependencies

```
http://maven.apache.org/maven-v4_0_0.xsd (xsi:schemaLocation with catalog)
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.demo</groupId>
  <artifactId>Maven_Sample_Project2</artifactId>
  <packaging>war</packaging>
  <version>0.0.1-SNAPSHOT</version>
  <name>Maven_Sample_Project2 Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.springframework/spring-webmvc -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>5.3.32</version>
    </dependency>

    <!-- https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>Maven_Sample_Project2</finalName>
  </build>
</project>
```

Step 2: Define the Controller

Create a controller class to handle requests.

```
package com.demo.controllers;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

@Controller
public class StudentController {

    @PostMapping("/add")
    public String addition(@RequestParam int num1, @RequestParam int num2)
    {
        int result=num1+num2;
        System.out.println("Addition= "+result);
        return "redirect:/welcome"; //redirect
    }

    @GetMapping("/welcome")
    public String wel() {
        return "welcome"; //welcome.jsp
    }
}
```

Step 3: Configure web.xml

Specify the DispatcherServlet in the web.xml file:

Bind to grammar/schema...

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">

    <servlet>
        <servlet-name>spring</servlet-name>
        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>spring</servlet-name>
        <url-pattern>/</url-pattern>
    </servlet-mapping>

</web-app>
```

Step 4: Define Bean Configuration (spring-servlet.xml)

Place the configuration in WEB-INF to enable component scanning and view resolution:

spring-servlet.xml:

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation) | http:
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         http://www.springframework.org/schema/beans/spring-beans.xsd
         http://www.springframework.org/schema/context
         http://www.springframework.org/schema/context/spring-context.xsd
         http://www.springframework.org/schema/mvc
         http://www.springframework.org/schema/mvc/spring-mvc.xsd">

    <context:component-scan base-package="com.demo.controllers"></context:component-scan>

    <!-- View Resolver -->
    <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
        <property name="prefix" value="/WEB-INF/pages/" />
        <property name="suffix" value=".jsp" />
    </bean>

    <!-- Enable Spring MVC -->
    <mvc:annotation-driven />

</beans>
```

Step 5: Create a JSP File

We will now create a JSP file to display the message.

index.jsp in /WEB-INF/pages/:

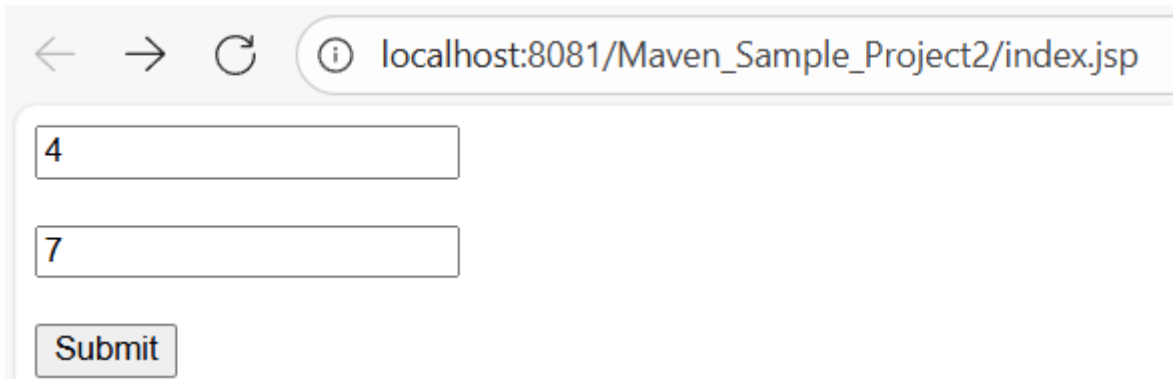
```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <h1>Welcome Page</h1>
</body>
</html>
```

This JSP file will display the message passed from the controller.

Step 6: Run the Application

Start the server and access the application:

index.jsp Page -



A screenshot of a web browser window. The address bar shows the URL 'localhost:8081/Maven_Sample_Project2/index.jsp'. The page content includes two text input fields. The first field contains the number '4' and the second field contains the number '7'. Below these fields is a button labeled 'Submit'.

console output –

```
Nov 18, 2025 9:39:36 PM org.apache.catalina.startup.Catalina start  
INFO: Server startup in [4836] milliseconds  
Addition= 11
```

welcome.jsp Page –



A screenshot of a web browser window. The address bar shows the URL 'localhost:8081/Maven_Sample_Project2/welcome'. The page content displays the text 'Welcome Page' in a large, bold, black serif font.

Advantages of Spring MVC :-

1. Lightweight and easy to deploy.
2. Supports rapid development.
3. Fast application development.
4. Multiple developers can work efficiently.
5. Easy to update applications.
6. Easier debugging with clear layers.

Disadvantages of Spring MVC :-

1. Complex for development.
2. Not suitable for small applications.